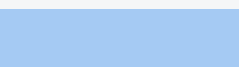
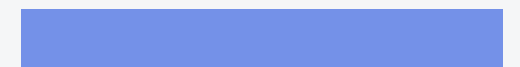
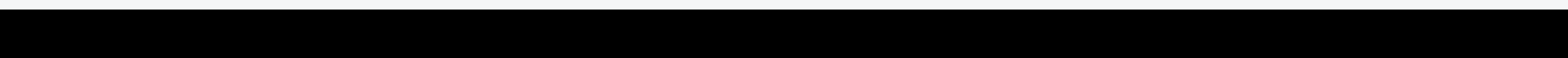


Lecture-6

Classes &

Objects

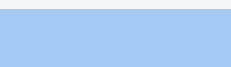
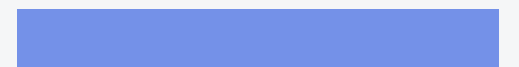
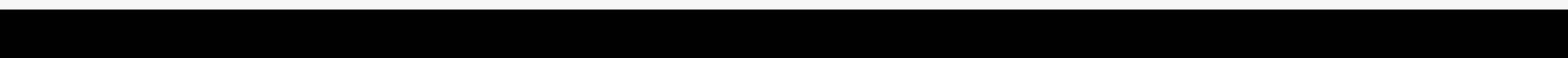


01. Structs

02. Class

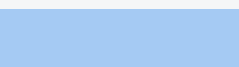
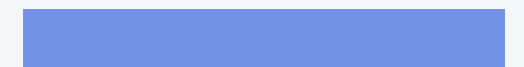
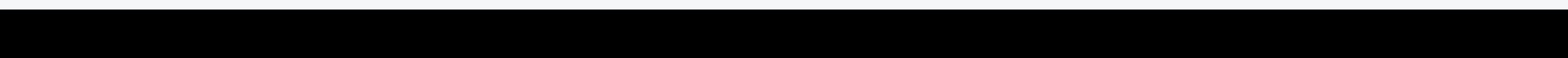
03. Objects

Agenda



01.

Structs



Structure

- A structure is a **collection of variables (c) and functions (c++)**.
- The variables in a structure can be of **different types**.
- The data items in a structure are called the **members of the structure**.

Struct

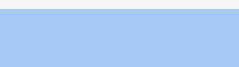
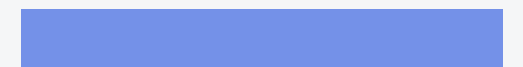
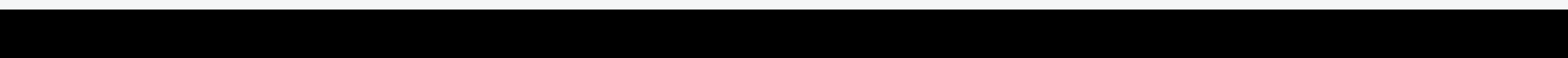
Only contains variables (c) and functions (c++). Variables can be of different data types.

Array

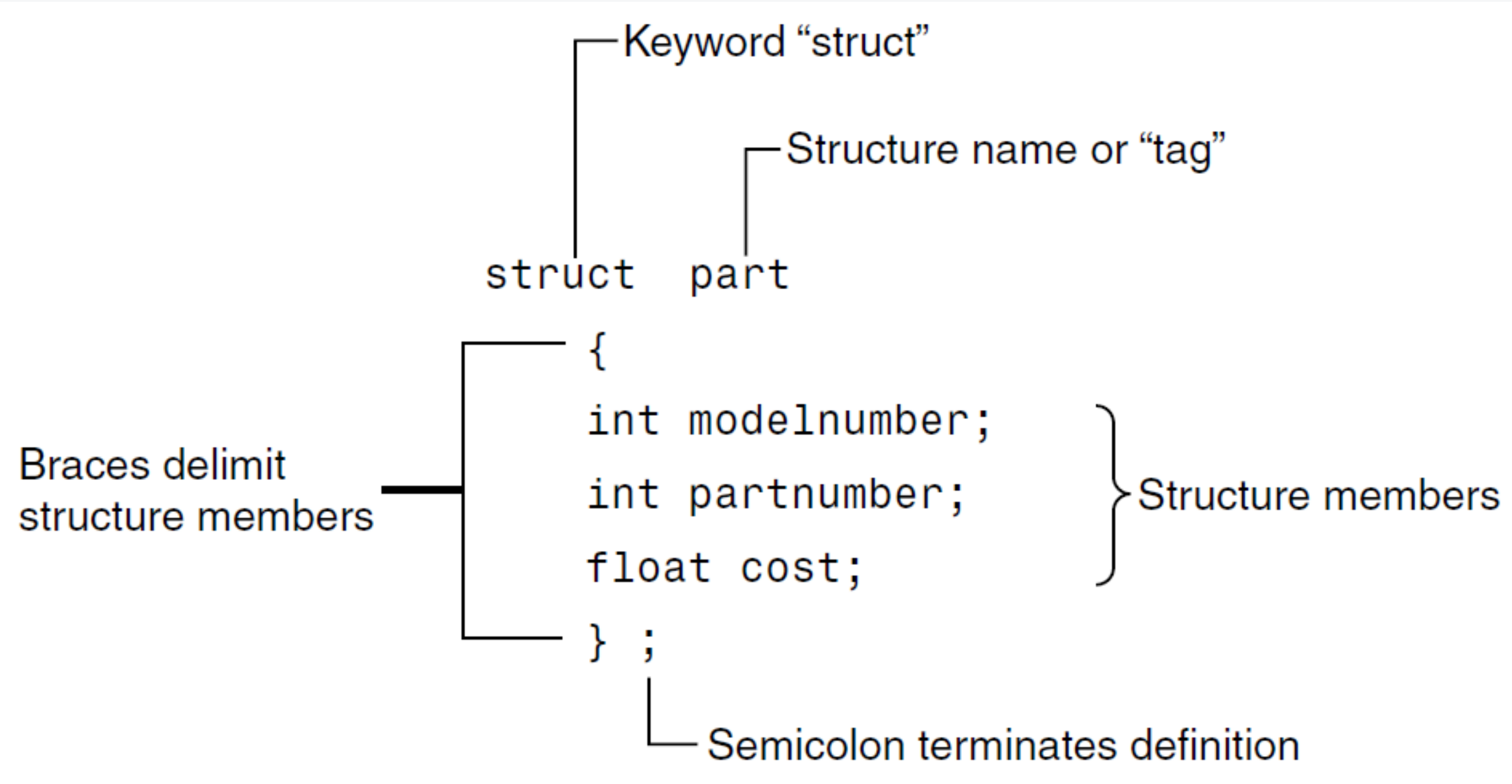
Only contains variables. Variables are of same data types.

Class

contains variables and functions. Variables can be of different data types.



Definition



Structure Variable

```
part part1;
```

Accessing Member

```
part1.modelnumber = 6244;
```


Initializing the struct variable

```
part part1;  
  
part1.modelnumber = 6244;  
part1.partnumber = 373;  
part1.cost = 217.55F;
```

Initializing the struct variable

```
part part1 = { 6244, 373, 217.55F };
```

Copying the Struct

```
part part1 = { 6244, 373, 217.55F };
```

```
part part2;
```

```
part2 = part1;
```

Members of Struct

- We can access the members of struct outside the struct.
- Their access level is public.
- So, is there a private access level?
- Answer is Yes.

Members of Struct

```
#include <iostream>
using namespace std;

struct part {
public:
    int modelnumber;
    int partnumber;
    float cost;
};

int main() {
    part p1;
    p1.modelnumber = 6244;
    p1.partnumber = 373;
    p1.cost = 217.55F;
}
```


Functions in Struct

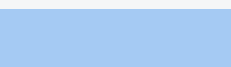
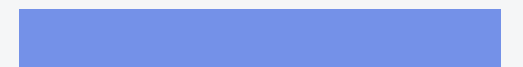
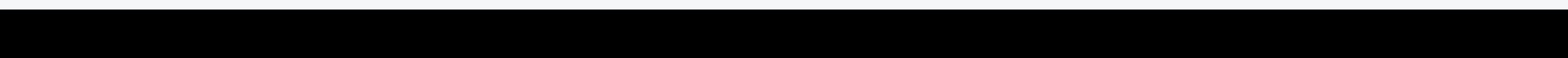
```
#include <iostream>
using namespace std;
struct part {
public:
    int modelnumber;
    int partnumber;
    float cost;
public:
    void display_cost() {
        cout<<"Cost of part number "<<partnumber<<" is "<< cost<<endl;
    }
};
int main() {
    part p1;
    p1.modelnumber = 6244;
    p1.partnumber = 373;
    p1.cost = 217.55F;
    p1.display_cost();
}
```

Takeaways

- **We have created struct with member data and member functions.**
- Member data and member functions can be public and private.
- In struct, by default member data and functions are public.

02.

Class



Struct to Class

- **Replace struct by class**
- In class, by default members are private.

Class

Member Data

Member Functions

Class Syntax

```
class foo
{
private:
    int data;
public:
    void memfunc (int d)
    { data = d; }
};
```

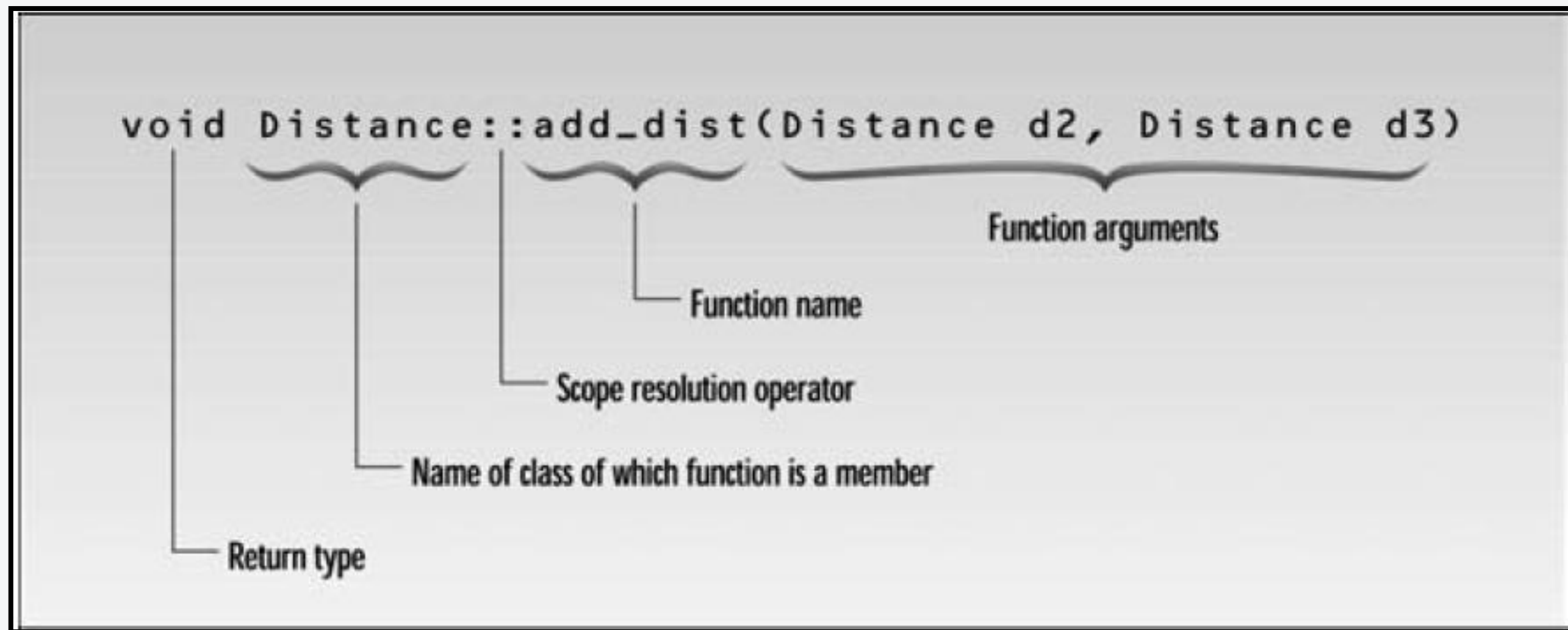
The diagram illustrates the syntax of a C++ class definition with the following annotations:

- Keyword:** Points to the `class` keyword.
- Name of class:** Points to the class name `foo`.
- Braces:** A bracket on the left side of the opening curly brace `{` points to the opening brace.
- Keyword private and colon:** Points to the `private:` access specifier.
- Private functions and data:** A bracket on the right side of the `private` block points to the `int data;` declaration.
- Keyword public and colon:** Points to the `public:` access specifier.
- Public functions and data:** A bracket on the right side of the `public` block points to the `void memfunc (int d) { data = d; }` definition.
- Semicolon:** Points to the closing brace and semicolon `};`.

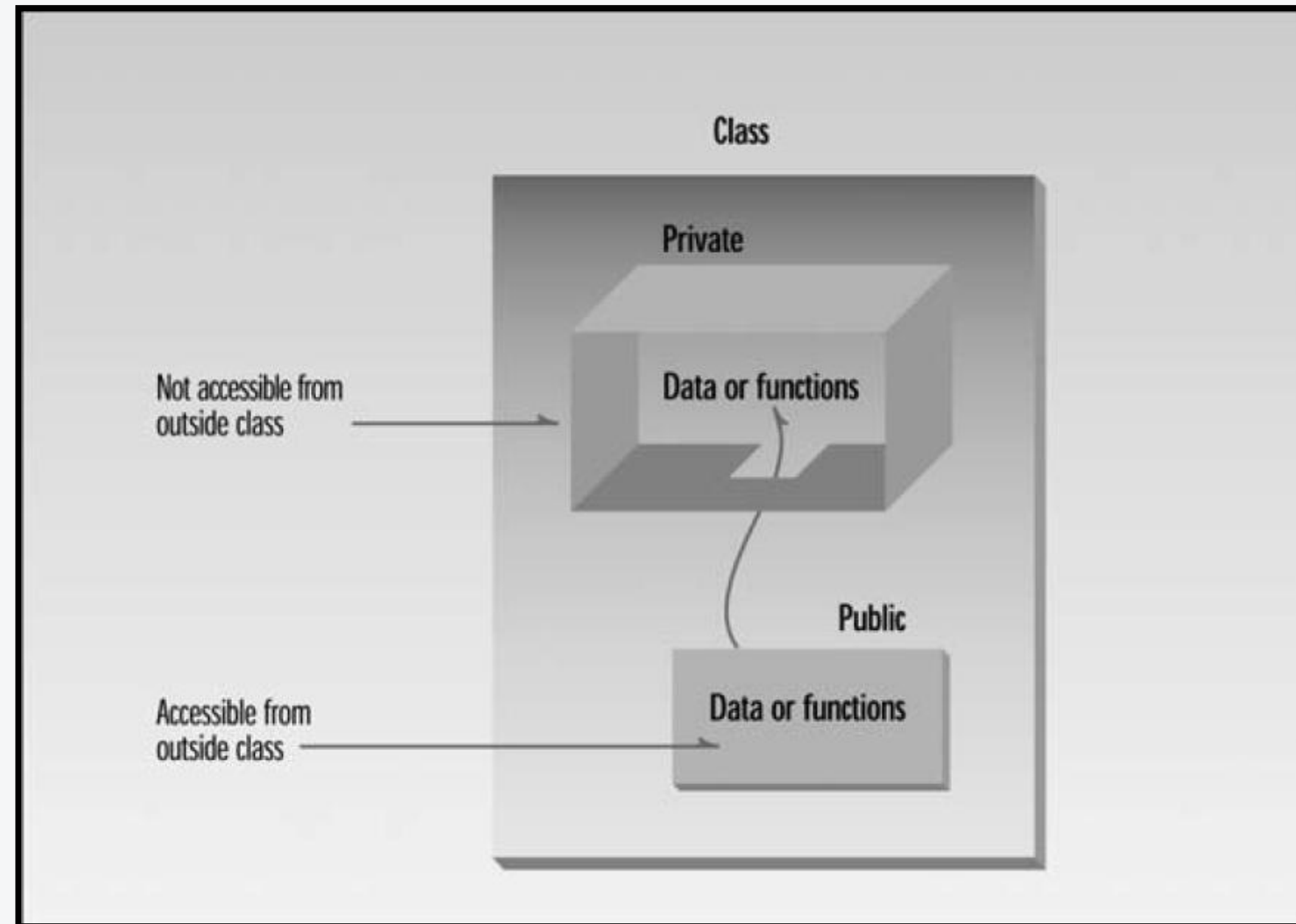
A Basic Class Code

```
class smallobj                //define a class
{
    private:
        int somedata;         //class data
    public:
        void setdata(int d)    //member function to set data
        { somedata = d; }
        void showdata()        //member function to display data
        { cout << "\nData is " << somedata; }
};
```

A Basic Class Code (Member Function O/S Class)

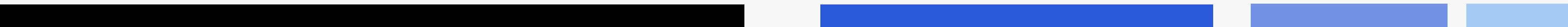


Data Hiding



03.

Objects/Instantiation



Objects

- Objects are instances of the class.
- Members can be accessed through objects.
- Dot operator is use for access. It is also called **class member access operator**.

Objects

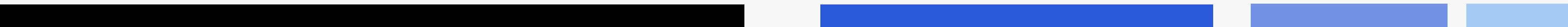
```
class smallobj                //define a class
{
private:
    int somedata;              //class data
public:
    void setdata(int d)        //member function to set data
        { somedata = d; }
    void showdata()             //member function to display data
        { cout << "\nData is " << somedata; }
};
```

```
smallobj s1, s2;
```

```
s1.setdata(1066);
s2.setdata(1776);
```

04.

Constructors



Constructors

Member functions

Automatic Initialization

No return type

Name same as class

Overloaded Constructors

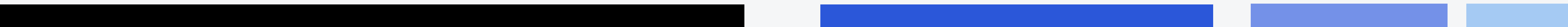


The Default Copy Constructor



05.

Objects and Functions



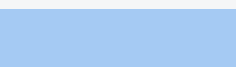
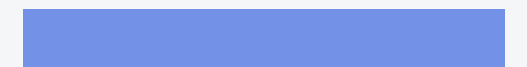
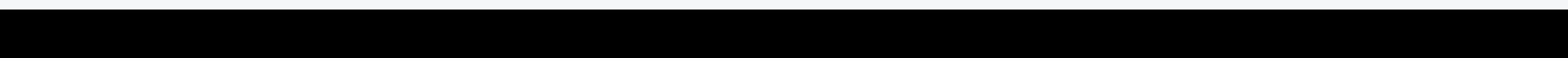
Objects as Function Arguments

Returning Objects from Functions



06.

Static Class Data



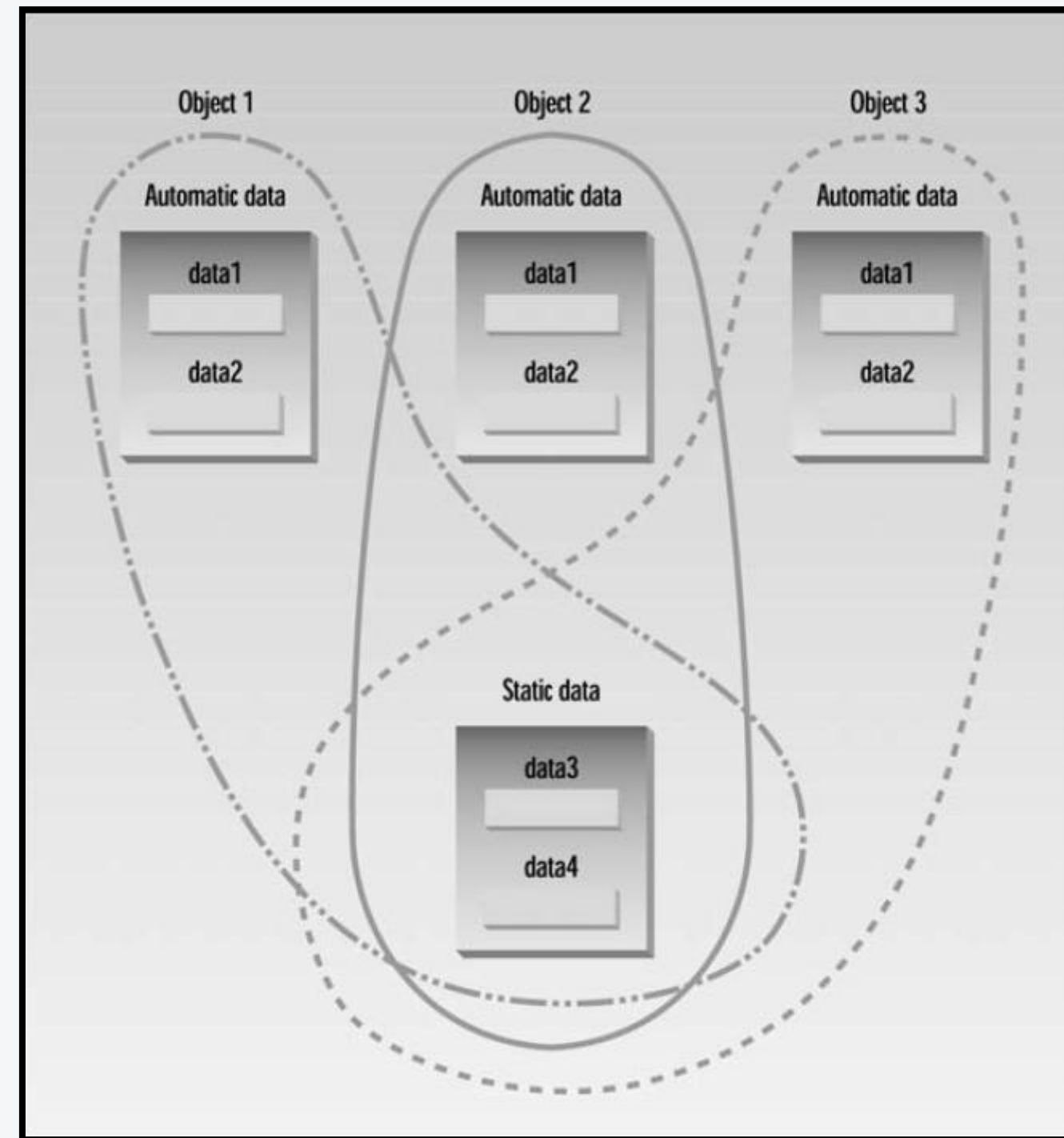
Static Class Data

- Each object contains its own separate data.
- A static data member is created for the entire class.
- A static data item is useful when all objects of the same class must share a common item of information.
- A static member variable is visible only within the class, but its lifetime is the entire program. It continues to exist even if there are no objects of the class.

Static Class Data

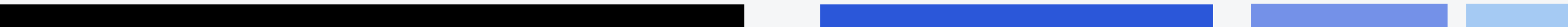
- Ordinary variables are usually declared (the compiler is told about their name and type) and defined (the compiler sets aside memory to hold the variable) in the same statement.
- Static member data requires two separate statements.
- The variable's declaration appears in the class definition, but the variable is actually defined outside the class, in much the same way as a global variable.

Static Class Data



07.

Const and Classes



const and Classes

- const Member Functions
- const Member Function Arguments
- const Objects

const Member Functions

- A const member function guarantees that it will never modify any of its class member data.
- A function is made into a constant function by placing the keyword const after the declarator but before the function body.
- If there is a separate function declaration, const must be used in both declaration and definition.

const Member Functions

```
class aClass
{
private:
    int alpha;
public:
    void nonFunc()          //non-const member function
        { alpha = 99; }    //OK

    void conFunc() const    //const member function
        { alpha = 99; }    //ERROR: can't modify a member
};
```


const Objects

- When an object is declared as const, you can't modify it.
- It follows that you can use only const member functions with it, because they're the only ones that guarantee not to modify it.

const Objects

```
#include <iostream>
using namespace std;
////////////////////////////////////
class Distance                                //English Distance class
{
private:
    int feet;
    float inches;
public:
    //2-arg constructor
    Distance(int ft, float in) : feet(ft), inches(in)
    { }
    void getdist()                            //user input; non-const func
    {
        cout << "\nEnter feet: ";  cin >> feet;
        cout << "Enter inches: ";  cin >> inches;
    }
    void showdist() const                    //display distance; const func
    { cout << feet << "'-" << inches << "'"; }
};
////////////////////////////////////
int main()
{
    const Distance football(300, 0);

    // football.getdist();                    //ERROR: getdist() not const
    cout << "football = ";
    football.showdist();                     //OK
    cout << endl;
    return 0;
}
```